데이터과학 파이썬

심송용(한림대학교 데이터과학스쿨)

http://jupiter.hallym.ac.kr

문자열

문자열의 개념

문자열은 이름, 주소 등과 같이 데이터를 처리할 때 자주 만나는 자료의 형태이다. 문자열은 시퀀스 자료형(sequence data type)으로 리스트처럼 데이터를 순차적으로 메모리에 저장하는 것을 말하나, 실제 메모리에 저장시에는 순서대로 저장하는 것은 아니고 사람이 눈으로 보기에 순차적이라는 의미이다.

문자열 자료는 앞에서 이미 본 것처럼, 큰 따옴표나 작은 따옴표로 표시한다.

문자열과 메모리 공간

- 컴퓨터에 문자를 저장하기 위해 ASCII(American Standard Code for Information Interchange) 방식을 표준으로 사용하기 시작. ASCII는 한 문자를 저장하기 위해 7bits ($2^7 = 128$)를 사용하는데 영문자, 숫자, 특수문자를 저장하는데 문제 없음.
- 로마자가 아닌 문자를 사용하는 컴퓨터에서 ASCII만으로 문자를 표현할 수 없음.

문자열

- Extended Unix Code (EUC)가 사용됨. EUC 주로 한국, 중국, 일본의 문자표현을 위해 사용하므로 euc-kr, euc-cn, euc-jp) 등으로 표시.
- 인터넷이 세계적으로 보편화되면 World Wide Web을 통한 전세계 문자를 하나의 인코딩으로 사용할 필요성이 생김
- UTF-8(Unicode Transformation Format-8): 최대 8bytes를 문자저장에 사용하여 어느 문자(eg: euc-kr, euc-jp)에 대한 인코딩인지 설정할 필요가 없음.
- MS Windows는 아직 CP949 인코딩을 사용하나 개개의 파일은 UTF-8을 사용하는 경우가 많음.

문자열의 저장과 인덱싱

- 파이썬에서 문자열은 각각의 문자를 원소로 하는 리스트.
- 따라서 리스트에서 사용하는 방법으로 인덱싱이 가능
- 슬라이싱도 같은 방법

문자열 'Hello'	**H**	**e**	**l**	**l**	**o**
0-양수 인덱스	0	1	2	3	4
음수 인덱스	-5	-4	-3	-2	-1

문자열

```
# strtest1.py
mystr0= '0123456789'
mystr1 = 'Hello. This is my string'
mystr2 = '안녕하세요? 내 문자열입니다'

print(mystr0[0], mystr0[2:4], mystr0[9])
print(mystr0[-1:-11:-1])
print(mystr1[0], mystr1[2:4], mystr1[9])
print(mystr2[0], mystr2[2:4], mystr2[9])
```

```
0 23 9
9876543210
H ll i
안 하세 문
```

문자열

문자열 연산과 함수

- + 문자열 합치기(연결)
- *: 문자열 반복
- in 특정문자열이 있는지 확인

```
# strtest2.py
mystr1 = 'Hello. This is my string'
mystr2 = '안녕하세요? 내 문자열입니다'

print(mystr1 + " 는 한글로 " + mystr2 + " 이다.")
print("\"" + mystr1 + "\" 는 한글로 " + "\"" + mystr2 + "\" 이다.")
print(mystr1 * 3)

if 'He' in mystr1:
    print('By first if: ' + mystr1)
```

문자열

```
else:
    print('By first if: ' + mystr2)

if '안녕' in mystr2:
    print('By second if: ' + mystr2)
else:
    print('By second if: ' + mystr1)
```

```
Hello. This is my string 는 한글로 안녕하세요? 내 문자열입니다 이다.
"Hello. This is my string" 는 한글로 "안녕하세요? 내 문자열입니다" 이다.
Hello. This is my stringHello. This is my stringHello. This is my string
By first if: Hello. This is my string
By second if: 안녕하세요? 내 문자열입니다
```

문자열

\로 시작하는 특수문자 표현(프린터용 \f, \r, \v)

\\	\ 표현	\"	"
\n	newline	\'	'
\t	tab	\b	backspace
\f	formfeed(새 페이지)	\r	carriage return(줄바꿈)
\xFF	FF에 해당하는 문자(00-FF까지 16진수)	\v	vertical tab

```
# strtest2.py (연속)
print("여기 줄 \n 빠꿉니다. \n \\로 backslash가 표현됩니다")
print("여기\t요기\t조기\t저기에 \'Tab\'\bAA")
print("\x22, \x13 ")
```

```
여기 줄
 빠꿉니다.
 \로 backslash가 표현됩니다
여기      요기      조기      저기에 'TabAA
", ◀
```

문자열

문자열 함수(메소드)

len(str)	str의 길이 반환
upper()	대문자로 변환
lower()	소문자로 변환
title()	각 단어의 첫글자는 대문자로
capitalize()	첫문자만 대문자로
strip()	시작과 끝부분의 빈칸 삭제
lstrip()	시작부분의 빈칸 삭제
rstrip()	끝부분의 빈칸 삭제
islower()	소문자인지 여부
isupper()	대문자인지 여부
split(sep)	sep 기준으로 문자열을 분리하여 리스트로 만들기. sep가 없으면 빈칸 기준 분리. 각 문자분리는 sep=' '로 설정(필요한지?? 이미 리스트)
isdigit()	숫자인지 판정
count(str)	str이 몇 번있는지 세기
find(str)	str이 왼쪽에서 몇 번째 문자인지 확인
rfind(str)	str이 오른쪽에서 몇 번째 문자인지 확인
startswith(str)	str로 문자열이 시작하는지 확인
endswith(str)	str로 문자열이 끝나는지 확인

문자열

```
# strtest3.py
mystr = '  hello. This is my string '

print("Length: " + str(len(mystr)))
print("Length: ", len(mystr.strip()))
print("Length: ", len(mystr.lstrip()))
print("Length: ", len(mystr.rstrip()))
print(mystr.lower())
print(mystr.upper())
print(mystr.title())
print(mystr.capitalize())
print(mystr.count('is'))
print(mystr.find('is'))
print(mystr.rfind('is'))
print(mystr.split())
print(mystr.split(' '))
```

```
Length: 27
Length:  24
Length:  25
Length:  26
  hello. this is my string
  HELLO. THIS IS MY STRING
  Hello. This Is My String
  hello. this is my string
2
11
14
['hello.', 'This', 'is', 'my', 'string']
['', '', 'hello.', 'This', 'is', 'my', 'string', '']
```